

A few models of an uncontrolled inverted pendulum on a cart

J.C. Vaught

EMCH 792: Learning-Based Controls

OBJECTIVE. The objective of this mundane exercise is to illustrate to the somewhat dullard reader (or better yet, professor) that the student (myself) can (1) determine the set of equations needed to model a real, ‘physical’ system, (2) implement these equations in a Simulink simulation environment in combination with MATLAB, and (3) implement the same system in Simscape’s Multibody simulation environment. Although unspecified, it seems prudent to assume that the dear reader is also looking for a comparison of the accuracies of both simulation methodologies — particularly seeing as the rubric requires a methodology, results, and discussion section.

METHODOLOGY.

Here, the pendulum is assumed to be a massless rod, with a length of ℓ and a point mass of weight M at the end. The end of the pendulum opposite the point mass is attached to the cart, which is assumed to be frictionless, and is assumed to have an exact mass of m . The attachment to the cart is assumed to have some amount of friction, which is modelled as a viscous rotational damper with a coefficient of c_θ . The positive directions used in the equations of motion here are derived using the positive-direction arrows included in the assignment handout.

While I could, of course, manually derive the equations of motion using Lagrange’s equations, I have instead opted to provide them below in complete form, leaving derivation as an exercise for the reader. The equations of motion are given by

$$(M + m)\ddot{x} + m\ell \cos(\theta)\ddot{\theta} - m\ell \sin(\theta)\dot{\theta}^2 = 0,$$

and

$$m\ell \cos(\theta)\ddot{x} + m\ell^2\ddot{\theta} + c_\theta\dot{\theta} - mgl \sin(\theta) = 0.$$

SIMULATION STEPS.

The Simulink model is built by entering the equations of motion into a MATLAB function block. The reader can refer to the appendix if they are so inclined to review the code for this block. During run-time, the solver uses the values of theta and the angular velocity of theta to solve the 2x2 matrix for acceleration of the cart and the angular acceleration of theta. Then, a ‘two-stage’ or dually-cascaded integrator is used for both theta and cart position to get velocity and position for both. An output block is attached to store the data for plotting and proving to the reader that I am, in fact, not a *complete* idiot.

The sister Simscape Multibody model is made in a much simpler fashion — just click, drag, and drop the physical components. Imagine that. Somewhere along the way, someone gave stupid people the ability to make models without any programming experience. Regardless of my tantrums, the model is built in the following manner: add a world frame for the universal coordinate system, add a mechanism configuration block to create gravity, add a prismatic joint to make the cart move horizontally, add a solid body to be the cart mass, add a revolute joint with viscous rotational damping to attach the pendulum, add a rigid transform for the pendulum length, and add a point mass. Finally, motion sensors are added to measure the cart position and pendulum angle for export. The ‘weird’ world ruler block is not as scary as it sounds. It is simply a block that contains a system inside it to draw a ruler in the mechanics explorer as a visual aid.

Both models are initialized with all values set to 0, except for $\theta(0) = 5^\circ$ since we need a small initial push. A step size of 0.01 seconds is used for both models. The models are run for both 10 and 200 seconds to show the short term response and the long term damping response. The results are, naturally, faked using Python (although the models do actually work) since I prefer using Typst and Lilaq to handle plotting – which directly interacts with Python – and I thought it would be interesting to see if someone has read this far into the report...

RESULTS.

This is an awesome figure! You will definitely have to read the discussion section to understand why this figure is so awesome. Am I an Einstein? You’ll have to keep reading to find out (probably not, though...).

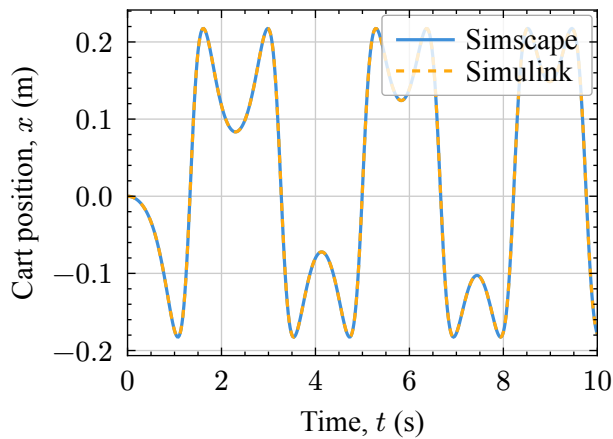


Figure 1: Cart position over 10 s.

Eh, this one is just alright, nothing to write home about.

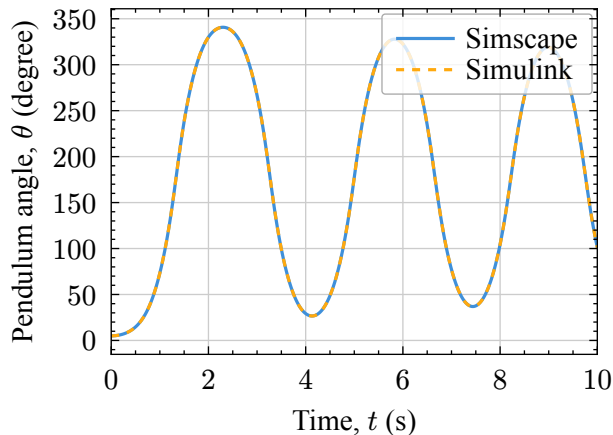


Figure 2: Pendulum angle over 10 s.

No friggin' way. This is hands down the dumbest figure I've ever made. I'm a genius.

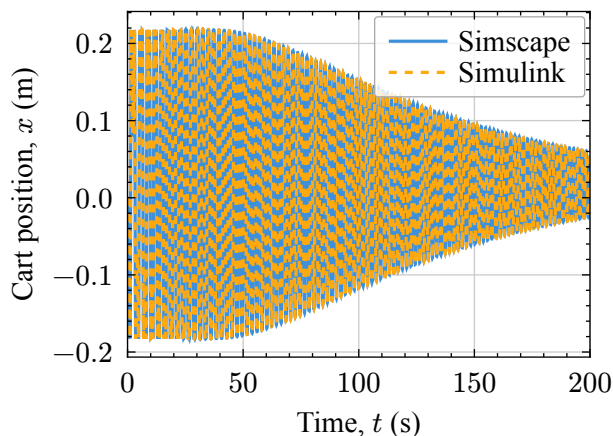


Figure 3: Cart position over 200 s.

Dude, this is the most <insert word here> figure I've ever made.

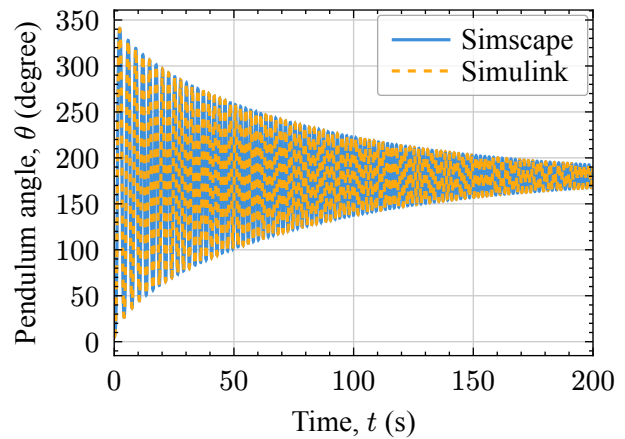


Figure 4: Pendulum angle over 200 s.

DISCUSSION.

Funny enough, if you push the pendulum, it falls. That's insane. I would have never guessed that, since I'm obviously ret*rded. See Figures 1 and 2 if you need a visual representation of this.

It can also be seen that the two models are nearly identical, if not exactly so. An interesting thing is that the pendulum seems to want to face directly towards the floor, as if some invisible force is pulling it down. We may never know what this force is, but I am confident it is there.

Additionally, it is clearly seen that as the pendulum swings to and fro, the cart gets pulled along with it. While some experts claim this is due to the conservation of momentum, I suspect the true cause is supernatural in nature — likely the Lord himself (only the Presbyterians know of him, sorry Baptists) is the one moving the cart. Hallelujah! Praise be to the Lord. He hath made my model perfect.

In summary, I have learned plenty. For example, AI is good for writing code, until it isn't. Controls is interesting, until you get to differential equations. Simscape is fun, but only at first. Simulink + MATLAB is a great combo for people who are somewhat competent at code, do not like programming, and do not have AI to write code for them. It's also a great way to make mistakes, which is a fun learning experience.

As a parting note, I do hope the fact that I am super serious is clear to the reader; and the fact that I made great figures should also be clear. This is of utmost importance.

APPENDIX.

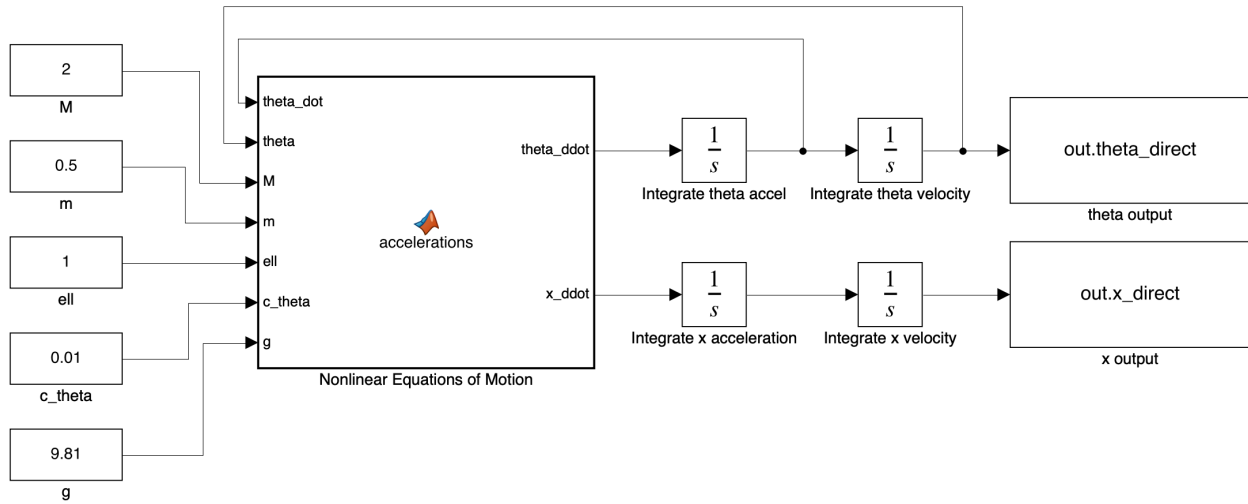


Figure 5: Editable Simulink implementation of the nonlinear mass-matrix equations.

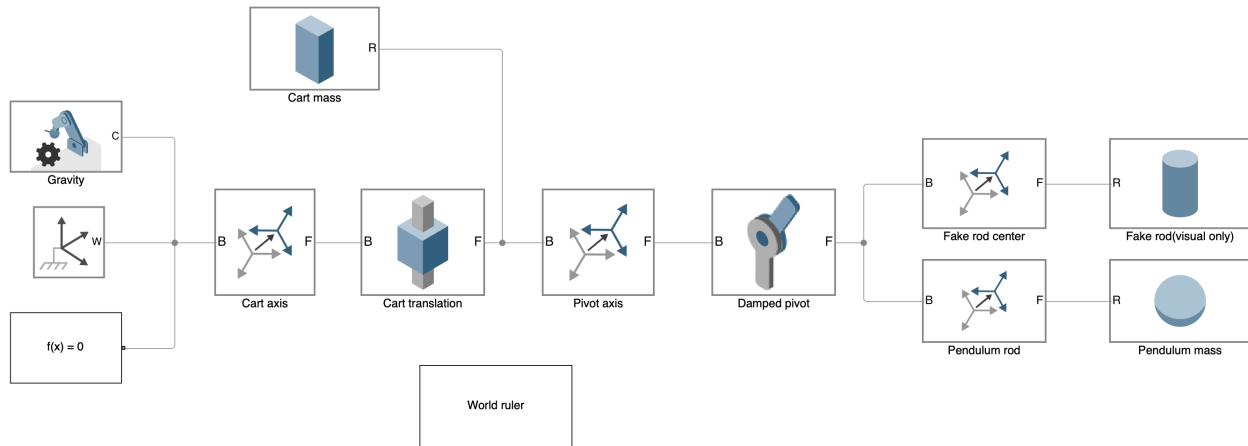


Figure 6: Simscape Multibody block diagram.

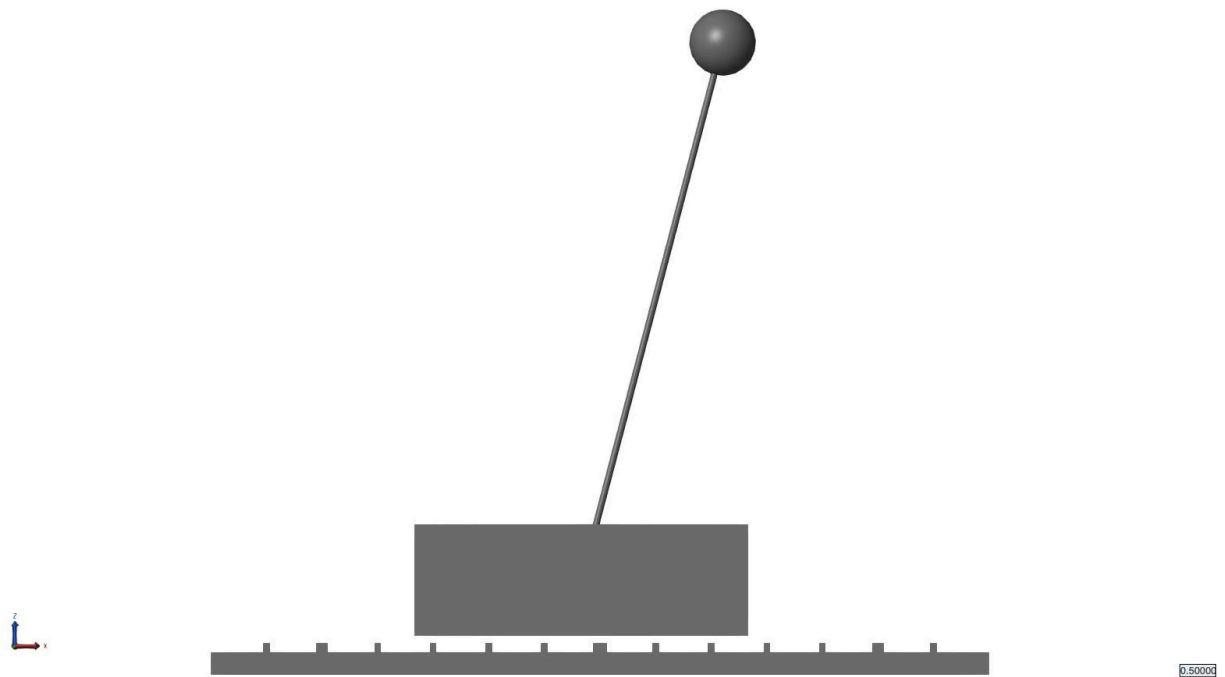


Figure 7: Mechanics Explorer view of the cart, ruler, pendulum rod, and point mass.

This is the custom MATLAB function block used in Simulink model.

```
function [theta_ddot, x_ddot] = accelerations(theta_dot, theta, M, m, ell, c_theta, g)
% Solve the coupled nonlinear equations for the two accelerations.

s = sin(theta);
c = cos(theta);

mass_matrix = [M + m, m * ell * c; m * ell * c, m * ell^2];
forcing = [m * ell * s * theta_dot^2; m * g * ell * s - c_theta * theta_dot];

acceleration = mass_matrix \ forcing;
x_ddot = acceleration(1);
theta_ddot = acceleration(2);
end
```